BIG DATA ANALYTICS - Homework Assignment 3

Q1: D3 Weather Visualizations and Stock Price Updates

Dataset and Data Preparation

For the first part of this assignment, I used the Seattle Weather dataset published by Vega, as directed in the assignment. It contains 1,461 daily weather observations recorded between January 1, 2012, and December 31, 2015.

The dataset includes six variables: date, precipitation, maximum temperature, minimum temperature, wind, and weather category.

I used D3.js version 7 to create the visualizations, with HTML providing the page structure, CSS handling presentation, and JavaScript processing the data and rendering the visualizations.

Before creating the charts, I loaded the CSV using d3.csv(), parsed dates with d3.utcParse(), and converted numerical values using Number(). This ensured the data could be used correctly in calculations and chart-generation functions.

1.1 Static Weather Visualizations

1.1.1 Wind Histogram: 10 Equal-Width Bins

The first visualization examines the distribution of recorded wind values using ten equal-width bins.

The bin width was calculated as:

$$\textit{Bin width} = (\textit{maximum wind} - \textit{minimum wind}) / 10$$

The observed wind values range from 0.4 to 9.5, producing a bin width of 0.91 source units.

I used d3.bin() with explicitly calculated boundaries to group observations. The horizontal axis represents wind values, and the vertical axis represents the number of days.

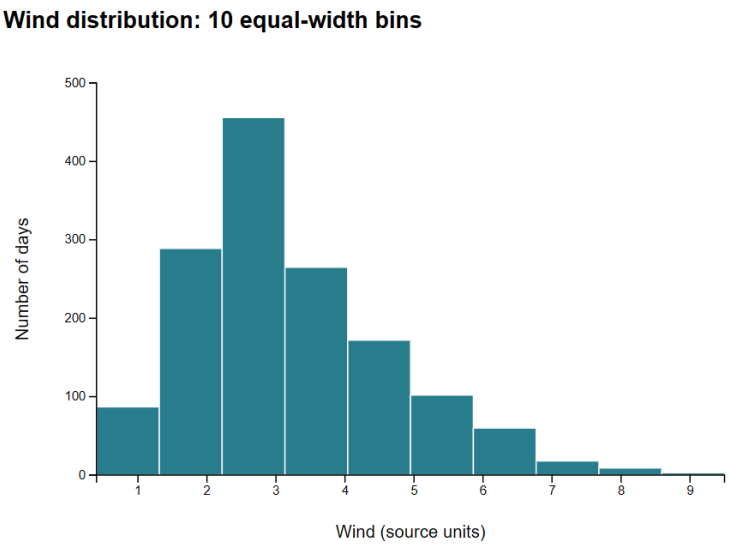


Figure 1. Wind distribution across 1,461 daily observations, using ten equal-width bins.

The histogram shows that wind observations are concentrated toward the lower portion of the recorded range. Specifically, 1,287 days, or approximately 88.1% of the dataset, have wind values at or below 5 source units.

1.1.2 Wind Histogram: 20 Equal-Width Bins

The second histogram uses the same dataset and horizontal-axis domain but increases the number of bins to twenty.

The updated bin width is: ***Bin width = (9.5 − 0.4) / 20 = 0.455 source units***

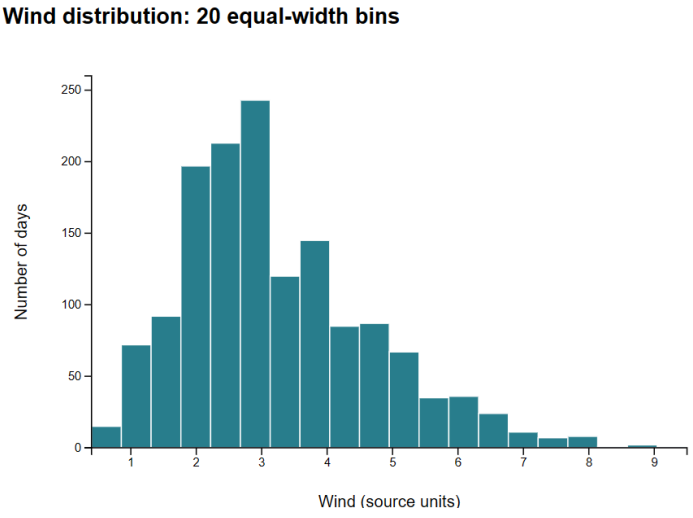


Figure 2. Wind distribution using twenty equal-width bins, allowing finer examination of local variation.

Comparing the two histograms demonstrates the effect of bin width on the visual representation of a distribution.

The ten-bin histogram offers a more aggregated view, making the overall distribution easier to recognize. The twenty-bin histogram reveals finer variation within the intervals.

Importantly, changing the number of bins does not change the observations being analyzed. Both histograms represent the same 1,461 days.

1.2.3 Weather Category Pie Chart

For the third visualization, I created a pie chart showing the distribution of five weather categories: rain, sun, fog, drizzle, and snow.

The category counts were calculated in JavaScript using d3.rollup(). Each percentage was computed as:

$$\textit{Category percentage} = 100 \times \textit{category count} / 1{,}461$$

The resulting frequencies were:

Weather category	Days	Percentage
Rain	641	43.9%
Sun	640	43.8%
Fog	101	6.9%
Drizzle	53	3.6%
Snow	26	1.8%
Total	**1,461**	**100%**

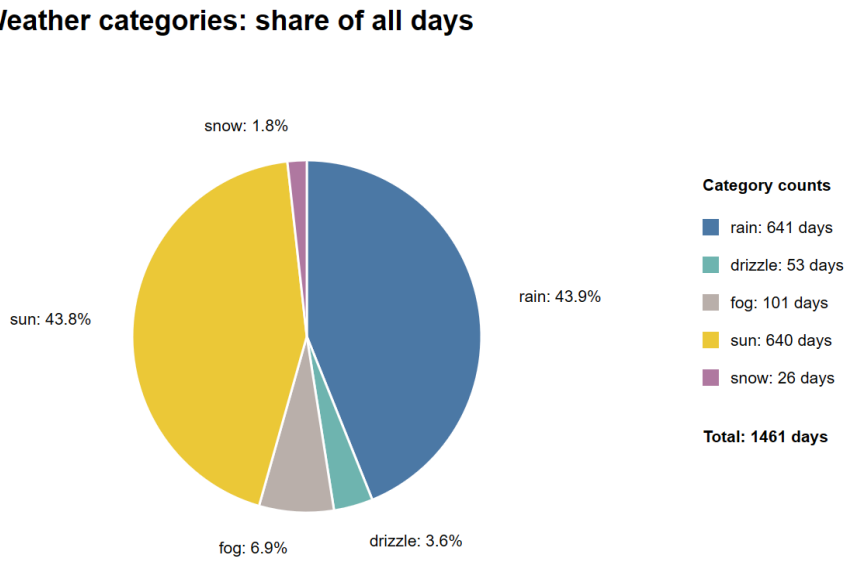


Figure 3. Distribution of weather categories, with category labels, percentages, and exact observation counts.

Rain and sun occurred at nearly identical frequencies, differing by only one day. Together, they account for approximately 87.7% of the dataset.

Snow was the least frequent recorded category, accounting for only 26 days.

1.1.4 Precipitation Time-Series Visualization

The fourth visualization examines daily precipitation over the complete four-year period.

I sorted the observations chronologically and used d3.scaleUtc() to map dates onto the horizontal axis. A linear scale mapped precipitation values to the vertical axis.

All 1,461 observations were included.

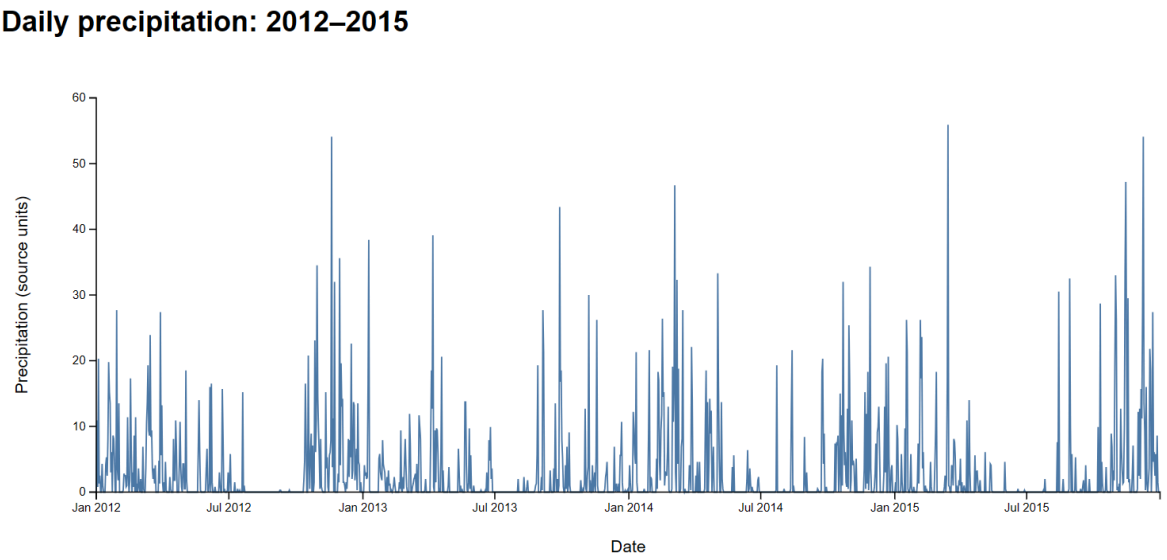


Figure 4. Daily precipitation between January 2012 and December 2015.

The precipitation series contains 838 days with zero recorded precipitation, approximately 57.4% of the observations.

The largest recorded daily precipitation value was 55.9 source units on March 15, 2015.

The visualization reveals isolated precipitation spikes alongside extended periods of low or zero precipitation.

1.2 Interactive Histogram

I extended the histogram implementation by adding two dropdown controls.

The first allows users to choose from five bin counts: 5, 10, 15, 20, and 25.

The second allows users to select one of four numerical variables: precipitation, maximum temperature, minimum temperature, or wind.

The chart updates dynamically without requiring a webpage reload.

Implementation

The selected variable determines which numerical values are passed to the histogram-generation function.

For every update, the function recalculates the minimum and maximum, derives equal-width bin boundaries, and updates the bars, axes, title, and labels.

Both dropdowns use the same update function, ensuring their selections remain synchronized.

When the variable changes, the existing bin count is preserved. The previous chart elements are removed before rendering the updated visualization, preventing duplicate bars or axes.

The two original static wind histograms remain available for comparison.

Interactive Demonstration

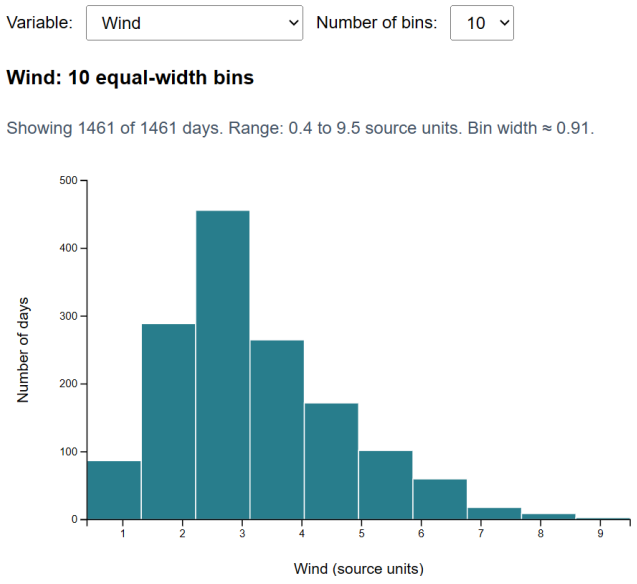


Figure 5. Initial interactive histogram using wind and ten bins.

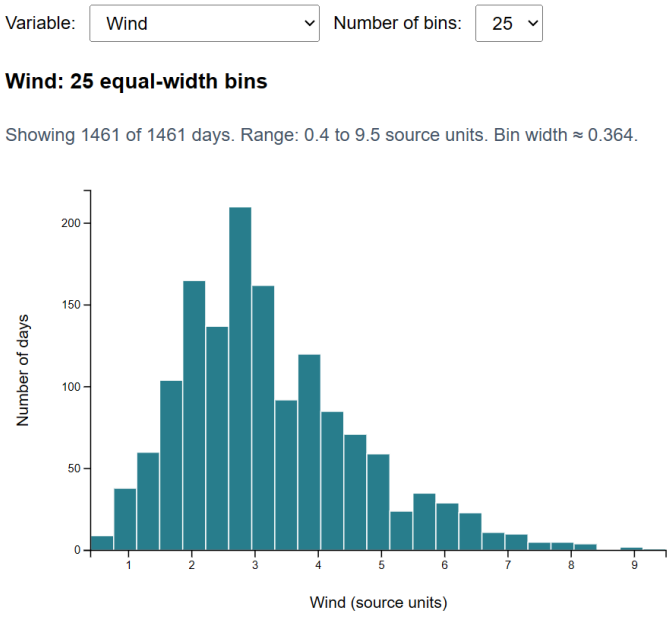


Figure 6. Interactive histogram after changing the bin count from ten to twenty-five.

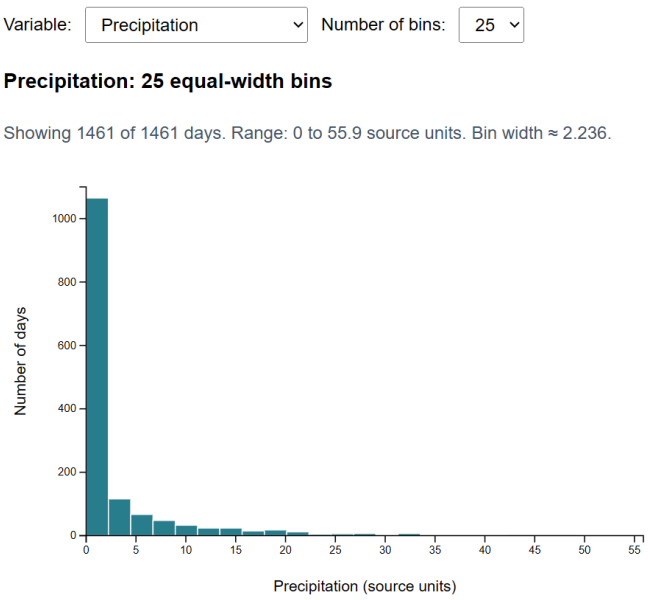


Figure 7. Interactive histogram after switching the variable to precipitation while preserving the selected bin count.

Observations

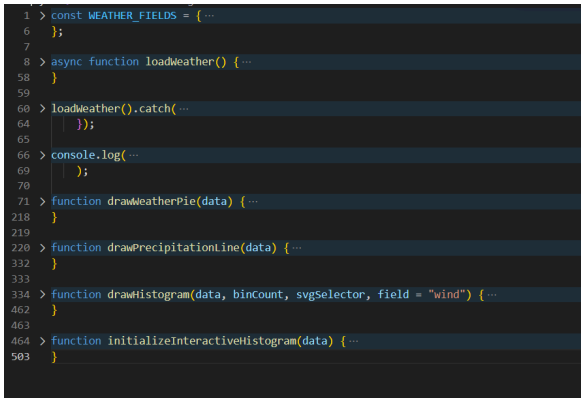
Observations on the full 2012–2015 dataset

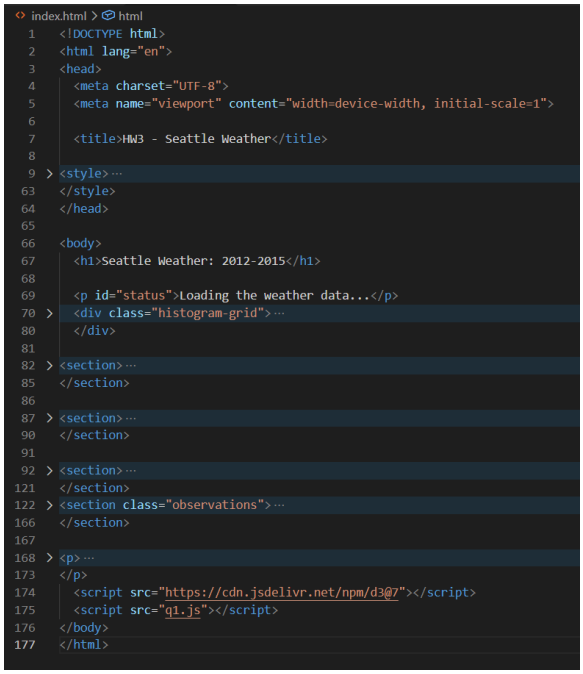
Across the 1,461 days, 1,287 (88.1%) have wind values at or below 5 source units. The ten-bin and twenty-bin histograms use identical observations and horizontal limits, but the twenty-bin version splits each wider interval in two and reveals finer local variation. In the weather pie, rain (641 days) and sun (640) together account for 87.7% of days; snow is least common with 26 days. The precipitation series has 838 zero-precipitation days, while the highest recorded daily value is 55.9 on March 15, 2015.

The interactive histogram makes it possible to inspect how precipitation, maximum temperature, minimum temperature, and wind differ in shape and range. Changing from ten to twenty-five bins reorganizes the same 1,461 observations into narrower groups; it does not change the data or sample size. Keeping the chosen bin count when switching variables helps readers compare distributions consistently, while retaining both static wind charts makes the original bin-width comparison easy to revisit.

Relevant Code

q1.js: index.html





Detailed codes:

```
// 1. Load the CSV.
const rows = await d3.csv("data/seattle-weather.csv");

// 2. Parse dates and convert numeric values.
const parseDate = d3.utcParse("%Y-%m-%d");

const weatherData = rows.map(row => ({
  date: parseDate(row.date),
  precipitation: Number(row.precipitation),
  temp_max: Number(row.temp_max),
  temp_min: Number(row.temp_min),
  wind: Number(row.wind),
  weather: row.weather
}));

// 3. Confirm that all required observations were loaded.
if (weatherData.length !== 1461) {
  throw new Error(
    `Expected 1461 rows, but loaded ${weatherData.length}.`
  );
}

console.log("Number of rows:", weatherData.length);
console.table(weatherData.slice(0, 5));

// 4. Calculate these counts after weatherData exists.
console.log(
  "Days with wind at or below 5:",
  weatherData.filter(day => day.wind <= 5).length
);

console.log(
  "Days with zero precipitation:",
  weatherData.filter(day => day.precipitation === 0).length
);
```

```
function drawHistogram(data, binCount, svgSelector, field = "wind") {
 // Read whichever numeric field was requested.
 const values = data.map(day => day[field]);

 const minValue = d3.min(values);
 const maxValue = d3.max(values);

 // Recalculate equal-width boundaries for this field.
 const binWidth = (maxValue - minValue) / binCount;

 const boundaries = d3.range(1, binCount).map(
   i => minValue + i * binWidth
 );

 const binGenerator = d3.bin()
   .domain([minValue, maxValue])
   .thresholds(boundaries);

 const bins = binGenerator(values);

 // Inspect the bins and check that every day is included.
 console.table(
   bins.map(bin => ({
     lowerBoundary: bin.x0,
     upperBoundary: bin.x1,
     numberOfDays: bin.length
   }))
 );

 console.log("Number of bins:", bins.length);
 console.log(
   "Days represented:",
   d3.sum(bins, bin => bin.length)
 );

 // 3. Set up the drawing area.
 const width = 600;
 const height = 400;
```

```
const margin = {
  top: 20,
  right: 20,
  bottom: 65,
  left: 75
};

const chartWidth = width - margin.left - margin.right;
const chartHeight = height - margin.top - margin.bottom;

const svg = d3.select(svgSelector)
  .attr("width", width)
  .attr("height", height)
  .attr("viewBox", `0 0 ${width} ${height}`)
  .style("max-width", "100%")
  .style("height", "auto");

// Clear this SVG before drawing into it.
svg.selectAll("*").remove();

const plot = svg.append("g")
  .attr(
    "transform",
    `translate(${margin.left}, ${margin.top})`
  );

// 4. Convert data values into positions on the screen.
const x = d3.scaleLinear()
  .domain([minValue, maxValue])
  .range([0, chartWidth]);

const y = d3.scaleLinear()
  .domain([0, d3.max(bins, bin => bin.length)])
  .nice()
  .range([chartHeight, 0]);
```

```
// 5. Draw one rectangle for each bin.
plot.selectAll("rect")
  .data(bins)
  .join("rect")
  .attr("x", bin => x(bin.x0))
  .attr("y", bin => y(bin.length))
  .attr("width", bin => x(bin.x1) - x(bin.x0))
  .attr("height", bin => chartHeight - y(bin.length))
  .attr("fill", "#287f8e")
  .attr("stroke", "white")
  .append("title")
  .text(bin =>
    `${bin.x0.toFixed(2)} to ${bin.x1.toFixed(2)}: ` +
    `${bin.length} days`
  );

// 6. Draw the horizontal axis.
plot.append("g")
  .attr("transform", `translate(0, ${chartHeight})`)
  .call(d3.axisBottom(x));

// 7. Draw the vertical axis.
plot.append("g")
  .call(
    d3.axisLeft(y)
      .ticks(6)
      .tickFormat(d3.format("d"))
  );

// 8. Label the horizontal axis.
plot.append("text")
  .attr("x", chartWidth / 2)
  .attr("y", chartHeight + 50)
  .attr("text-anchor", "middle")
  .attr("font-size", 14)
  .text(`${WEATHER_FIELDS[field]} (source units)`);
```

```
// 9. Label the vertical axis.
plot.append("text")
  .attr("transform", "rotate(-90)")
  .attr("x", -chartHeight / 2)
  .attr("y", -55)
  .attr("text-anchor", "middle")
  .attr("font-size", 14)
  .text("Number of days");

  return {
binCount: bins.length,
count: d3.sum(bins, bin => bin.length),
min: minValue,
max: maxValue,
binWidth
};
}
```

```
function initializeInteractiveHistogram(data) {
  const variableSelect = d3.select("#histogram-variable");
  const binSelect = d3.select("#histogram-bins");

  const formatNumber = d3.format(".3~f");

  function update() {
    // Read both current selections.
    const field = variableSelect.property("value");
    const binCount = Number(binSelect.property("value"));

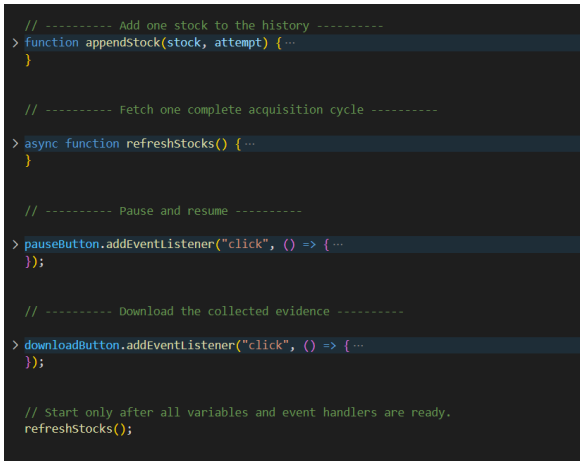
    // Draw the chart and receive its calculated summary.
    const summary = drawHistogram(
      data,
      binCount,
      "#histogram-dynamic",
      field
    );

    d3.select("#histogram-title")
      .text(
        `${WEATHER_FIELDS[field]}: ` +
        `${summary.binCount} equal-width bins`
      );

    d3.select("#histogram-summary")
      .text(
        `Showing ${summary.count} of ${data.length} days. ` +
        `Range: ${summary.min} to ${summary.max} source units. ` +
        `Bin width ≈ ${formatNumber(summary.binWidth)}.`
      );
  }

  // Run the same update whenever either dropdown changes.
  variableSelect.on("change", update);
  binSelect.on("change", update);

  // Draw the initial chart when the data finishes loading.
  update();
}
```



```
// ---------- Add one stock to the history ----------
function appendStock(stock, attempt) {…
}


// ---------- Fetch one complete acquisition cycle ----------

async function refreshStocks() {…
}


// ---------- Pause and resume ----------

pauseButton.addEventListener("click", () => {…
});


// ---------- Download the collected evidence ----------

downloadButton.addEventListener("click", () => {…
});


// Start only after all variables and event handlers are ready.
refreshStocks();
```



```
  statusText.textContent =
    `Cycle ${attempt.cycle}: requesting stock data...`;

  scheduleText.textContent = "Request in progress.";
  errorText.hidden = true;

  try {
    const response = await fetch("/api/stocks", {
      cache: "no-store"
    });

    attempt.http_status = response.status;

    const responseText = await response.text();

    try {
      attempt.snapshot = JSON.parse(responseText);
    } catch {
      attempt.response_text = responseText;

      throw new Error(
        `HTTP ${response.status}: expected JSON. ` +
        "Check that you opened stocks.html using port 8001."
      );
    }
```

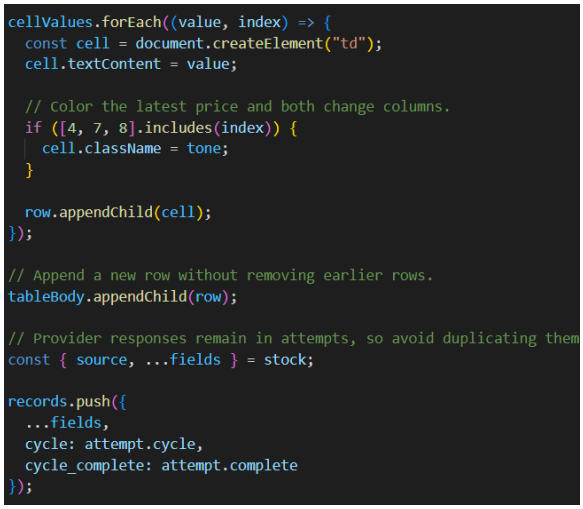


```
function appendStock(stock, attempt) {
  const tone =
    stock.change > 0 ? "positive" :
    stock.change < 0 ? "negative" :
    "neutral";

  const direction =
    stock.change > 0 ? "Up" :
    stock.change < 0 ? "Down" :
    "No change";

  const cycleLabel = attempt.complete
    ? String(attempt.cycle)
    : `${attempt.cycle} (partial)`;

  const cellValues = [
    cycleLabel,
    formatUtc(stock.retrieved_at_utc),
    stock.ticker,
    stock.latest_date,
    stock.latest_price.toFixed(2),
    stock.previous_close_date,
    stock.previous_close.toFixed(2),
    `${direction} ${signedNumber(stock.change)}`,
    `${signedNumber(stock.percent_change)}%`
  ];
```



```
cellValues.forEach((value, index) => {
  const cell = document.createElement("td");
  cell.textContent = value;

  // Color the latest price and both change columns.
  if ([4, 7, 8].includes(index)) {
    cell.className = tone;
  }

  row.appendChild(cell);
});

// Append a new row without removing earlier rows.
tableBody.appendChild(row);

// Provider responses remain in attempts, so avoid duplicating them
const { source, ...fields } = stock;

records.push({
  ...fields,
  cycle: attempt.cycle,
  cycle_complete: attempt.complete
});
```

1.4 Periodic Stock Price Updates

The final part of Q1 required a webpage displaying periodically updated stock prices.

I selected three stocks: Apple (AAPL), Microsoft (MSFT), and Alphabet (GOOGL).

The webpage retrieves pricing information through the Massive Stocks API, using a local Python server to handle API authentication and requests.

Data Acquisition

I used end-of-day daily price aggregates supported by the API plan. For each stock, the server retrieves the latest available completed trading-session closing price and the closing price from the preceding trading session.

The application preserves both provider trading dates and the webpage's retrieval timestamp. This distinction is important because repeated requests can return identical end-of-day prices.

Price Comparison

For every stock, the absolute change is:

$$\textit{Price change} = \textit{latest close} - \textit{previous close}$$

The percentage change is:

$$\textit{Percentage change} = 100 \times (\textit{latest close} - \textit{previous close}) / \textit{previous close}$$

The webpage uses green text for positive changes, red text for negative changes, and a neutral label for unchanged prices. A signed numerical value or direction label accompanies each result.

Periodic Refresh

The application schedules a new update 90 seconds after the preceding cycle finishes. The Python backend spaces provider requests by at least 13 seconds to respect the API's request limits. Each successful update appends new rows to the existing history rather than replacing previously displayed records.

Because the data are end-of-day observations, a refresh may retrieve the same closing prices. Such repetitions are retained as separate acquisition records with distinct retrieval timestamps.

Results

Successful cycles: 5 | Rows: 15

End-of-day daily closes; split-adjusted. Requested date range: 2026-09-20 through 2026-10-20.

Acquisition history — prices in USD; retrieval times in UTC

Cycle	Retrieved (UTC)	Ticker	Latest date (ET)	Latest close (USD)	Previous date (ET)	Previous close (USD)	Change (USD)	Change (%)
1	2026-10-21 20:51:53	AAPL	2026-10-20	339.50	2026-10-19	342.94	Down -3.44	-1.00%
1	2026-10-21 20:52:19	MSFT	2026-10-20	531.87	2026-10-19	533.52	Down -1.65	-0.31%
1	2026-10-21 20:52:32	GOOGL	2026-10-20	340.27	2026-10-19	344.18	Down -3.91	-1.13%
2	2026-10-21 20:54:03	AAPL	2026-10-20	339.50	2026-10-19	342.94	Down -3.44	-1.00%
2	2026-10-21 20:54:16	MSFT	2026-10-20	531.87	2026-10-19	533.52	Down -1.65	-0.31%
2	2026-10-21 20:54:29	GOOGL	2026-10-20	340.27	2026-10-19	344.18	Down -3.91	-1.13%
3	2026-10-21 20:55:59	AAPL	2026-10-20	339.50	2026-10-19	342.94	Down -3.44	-1.00%
3	2026-10-21 20:56:12	MSFT	2026-10-20	531.87	2026-10-19	533.52	Down -1.65	-0.31%
3	2026-10-21 20:56:25	GOOGL	2026-10-20	340.27	2026-10-19	344.18	Down -3.91	-1.13%
4	2026-10-21 20:57:56	AAPL	2026-10-20	339.50	2026-10-19	342.94	Down -3.44	-1.00%
4	2026-10-21 20:58:09	MSFT	2026-10-20	531.87	2026-10-19	533.52	Down -1.65	-0.31%
4	2026-10-21 20:58:22	GOOGL	2026-10-20	340.27	2026-10-19	344.18	Down -3.91	-1.13%
5	2026-10-21 20:59:53	AAPL	2026-10-20	339.50	2026-10-19	342.94	Down -3.44	-1.00%
5	2026-10-21 21:00:06	MSFT	2026-10-20	531.87	2026-10-19	533.52	Down -1.65	-0.31%
5	2026-10-21 21:00:19	GOOGL	2026-10-20	340.27	2026-10-19	344.18	Down -3.91	-1.13%

Figure 8. Periodically updated stock-price history showing all three tickers, provider trading dates, webpage retrieval timestamps, and color-coded changes.

The recorded history demonstrates that the webpage can request stock prices periodically, calculate price differences, and preserve previous results without reloading. The displayed data are labeled as end-of-day observations rather than live intraday prices.

Relevant Code

```
// ---------- Add one stock to the history ----------
function appendStock(stock, attempt) {
  const tone =
    stock.change > 0 ? "positive" :
    stock.change < 0 ? "negative" :
    "neutral";

  const direction =
    stock.change > 0 ? "Up" :
    stock.change < 0 ? "Down" :
    "No change";

  const cycleLabel = attempt.complete
    ? String(attempt.cycle)
    : `${attempt.cycle} (partial)`;

  const cellValues = [
    cycleLabel,
    formatUtc(stock.retrieved_at_utc),
    stock.ticker,
    stock.latest_date,
    stock.latest_price.toFixed(2),
    stock.previous_close_date,
    stock.previous_close.toFixed(2),
    `${direction} ${signedNumber(stock.change)}`,
    `${signedNumber(stock.percent_change)}%`
  ];

  const row = document.createElement("tr");

  cellValues.forEach((value, index) => {
    const cell = document.createElement("td");
    cell.textContent = value;

    // Color the latest price and both change columns.
    if ([4, 7, 8].includes(index)) {
      cell.className = tone;
    }
    row.appendChild(cell);
  });

  // Append a new row without removing earlier rows.
  tableBody.appendChild(row);

  // Provider responses remain in attempts, so avoid duplicating them.
  const { source, ...fields } = stock;

  records.push({
    ...fields,
    cycle: attempt.cycle,
    cycle_complete: attempt.complete
  });
}
```

```
// ---------- Fetch one complete acquisition cycle ----------

async function refreshStocks() {
  if (paused || inFlight) {
    return;
  }

  inFlight = true;
  clearTimeout(timerId);
  timerId = null;

  const attempt = {
    cycle: ++cycleNumber,
    started_at_utc: new Date().toISOString(),
    finished_at_utc: null,
    http_status: null,
    complete: false,
    records_added: 0,
    problems: [],
    snapshot: null
  };

  statusText.textContent =
    `Cycle ${attempt.cycle}: requesting stock data...`;

  scheduleText.textContent = "Request in progress.";
  errorText.hidden = true;

  try {
    const response = await fetch("/api/stocks", {
      cache: "no-store"
    });

    attempt.http_status = response.status;

    const responseText = await response.text();
    try {
      attempt.snapshot = JSON.parse(responseText);
    } catch {
      attempt.response_text = responseText;

      throw new Error(
        `HTTP ${response.status}: expected JSON. ` +
        "Check that you opened stocks.html using port 8001."
      );
    }

    if (!response.ok) {
      throw new Error(`Server returned HTTP ${response.status}.`);
    }

    const snapshot = attempt.snapshot;

    if (
      !snapshot ||
      !Array.isArray(snapshot.stocks) ||
      !Array.isArray(snapshot.errors)
    ) {
      throw new Error("The server returned an unexpected data structure.");
    }

    windowText.textContent =
      `${snapshot.mode}. Requested date range: ` +
      `${snapshot.requested_from} through ${snapshot.requested_through}.`;

    attempt.problems.push(
      ...snapshot.errors.map(error =>
        `${error?.ticker ?? "API"}: ` +
        `${error?.message ?? "Unknown error"}`
      )
    );

    const seenTickers = new Set();
    const validStocks = [];
```

```
for (const stock of snapshot.stocks) {
  if (
    !isValidStock(stock) ||
    seenTickers.has(stock.ticker)
  ) {
    attempt.problems.push(
      `Skipped an invalid or duplicate record: ` +
      `${stock?.ticker ?? "unknown ticker"}.`
    );

    continue;
  }

  seenTickers.add(stock.ticker);
  validStocks.push(stock);
}

const missingTickers = EXPECTED_TICKERS.filter(
  ticker => !seenTickers.has(ticker)
);

if (missingTickers.length > 0) {
  attempt.problems.push(
    `Missing valid records for: ${missingTickers.join(", ")}.`
  );
}

if (snapshot.ok !== true) {
  attempt.problems.push(
    "The server did not report a complete successful cycle."
  );
}

attempt.complete =
  snapshot.ok === true &&
  attempt.problems.length === 0 &&
  validStocks.length === EXPECTED_TICKERS.length;
```

```
  for (const stock of validStocks) {
    appendStock(stock, attempt);
  }

  attempt.records_added = validStocks.length;

  if (attempt.complete) {
    successfulCycles += 1;

    statusText.textContent =
      `Cycle ${attempt.cycle} complete: ` +
      `added ${validStocks.length} stock rows.`;
  } else {
    statusText.textContent =
      `Cycle ${attempt.cycle} incomplete: ` +
      `added ${validStocks.length} valid stock rows.`;
  }
} catch (error) {
  attempt.problems.push(
    error instanceof Error ? error.message : String(error)
  );

  statusText.textContent =
    `Cycle ${attempt.cycle} failed. Earlier rows are retained.`;
} finally {
  attempt.finished_at_utc = new Date().toISOString();
  attempts.push(attempt);

  inFlight = false;

  countsText.textContent =
    `Successful cycles: ${successfulCycles} | Rows: ${records.length}`;

  errorText.textContent = attempt.problems.join("\n");
  errorText.hidden = attempt.problems.length === 0;

  downloadButton.disabled = false;
```

```
  downloadButton.disabled = false;

  if (paused) {
    scheduleText.textContent = "Updates paused.";
  } else {
    const nextRequestTime = Date.now() + REFRESH_DELAY_MS;

    scheduleText.textContent =
      `Next request starts around ${formatUtc(nextRequestTime)} UTC.`;

    timerId = setTimeout(refreshStocks, REFRESH_DELAY_MS);
  }
 }
}
```

```
pauseButton.addEventListener("click", () => {
  paused = !paused;

  clearTimeout(timerId);
  timerId = null;

  pauseButton.textContent = paused
    ? "Resume updates"
    : "Pause updates";

  if (paused) {
    scheduleText.textContent = inFlight
      ? "Pausing after the current request finishes."
      : "Updates paused.";
  } else if (inFlight) {
    scheduleText.textContent = "Request in progress.";
  } else {
    refreshStocks();
  }
});
```

```
downloadButton.addEventListener("click", () => {
  const evidence = {
    exported_at_utc: new Date().toISOString(),
    source: "Massive daily stock aggregates",
    expected_tickers: EXPECTED_TICKERS,
    price_mode: "End-of-day daily closes; split-adjusted",
    comparison:
      "Latest returned daily close minus previous returned trading close.",
    refresh_policy: {
      delay_after_completion_ms: REFRESH_DELAY_MS,
      overlapping_requests: false
    },
    successful_cycles: successfulCycles,
    row_count: records.length,
    records,
    attempts
  };

  const blob = new Blob(
    [JSON.stringify(evidence, null, 2)],
    { type: "application/json" }
  );

  const url = URL.createObjectURL(blob);
  const link = document.createElement("a");

  link.href = url;
  link.download = "q1_stock_evidence.json";

  document.body.appendChild(link);
  link.click();
  link.remove();

  setTimeout(() => URL.revokeObjectURL(url), 1000);
});
```

Saved evidence: q1_stock_evidence.json

Q2: Stock Data Storytelling

2.1 Audience, Focused Question, and Big Idea

Audience: A risk-monitoring manager responsible for NVDA, TSLA, and SPY, with the capacity to manually review at most one asset per trading day.

Focused Question: How would a 3% versus 5% daily-swing alert threshold affect monitoring coverage and review workload between August 21 and October 2, 2026?

Big Idea: A 3% daily-swing threshold identifies 14 review dates rather than three at 5%, so piloting a largest-swing-first review rule would broaden screening coverage within the one-asset-per-day limit, while requiring more review time.

2.2 Evidence

I retrieved daily historical stock data for NVDA, TSLA, and SPY from Yahoo Finance using the Python yfinance library. The dataset covers August 21 through October 2, 2026, with 30 trading days for each asset, producing 90 observations.

The download used these settings:

- Start date: August 21, 2026
- Exclusive end date: October 3, 2026
- Interval: One day
- auto_adjust=False
- back_adjust=False
- repair=False
- rounding=False
- prepost=False
- actions=False

Each observation includes the trading date, ticker, Open, High, Low, Close, and Volume. The raw provider observations were preserved, and processed results were saved separately.

Calculated Metrics

The normalized closing-price index was defined as:

$$\textit{Normalized index on date } t = 100 \times \textit{Close on date } t \,/\, \textit{Close on August 21}$$

The return over the assigned window was:

$$\textit{Window return (\%)} = 100 \times (\textit{October 2 Close} - \textit{August 21 Close}) \,/\, \textit{August 21 Close}$$

The daily price swing was:

$$\textit{Daily swing (\%)} = 100 \times (\textit{High} - \textit{Low}) \,/\, \textit{Open}$$

Average daily swing was calculated as the arithmetic mean of the thirty daily swings for each ticker. Two common alert thresholds, 3% and 5%, were evaluated across the entire period.

I chose 3% as a lower exploratory screening threshold and 5% as a stricter alternative to compare broader monitoring coverage with a smaller review workload. I applied both values to all three assets so that alert counts are directly comparable. These thresholds are illustrative

choices rather than optimized risk limits or estimates of the probability of loss. An asset triggers an alert when its daily swing is greater than or equal to the selected threshold.

Numerical precision was preserved during calculations, with rounding applied only when displaying results.

Stock	Window Return	Average Daily Swing	Alerts at 3%	Alerts at 5%
NVDA	+8.96%	2.37%	6	1
TSLA	+2.13%	3.17%	11	2
SPY	+0.51%	0.67%	0	0

Table 1. Full-period comparison of the three monitored assets.

Monitoring Workload

Threshold	Asset Alerts	Dates With Alerts	Dates With Competing Alerts	Alerts Without Separate Review
3%	17	14	3	3
5%	3	3	0	0

Table 2. Alert frequency and review workload under both threshold settings.

The distinction between asset alerts and dates with alerts is important because multiple assets can trigger alerts on the same trading day.

2.3 Evidence Views

View 1: Full-Window Stock Comparison

1. TSLA triggered the most alerts despite NVDA's higher return

Ticker	Window return	Average daily swing	3% alerts	5% alerts
NVDA	8.96%	2.37%	6	1
TSLA	2.13%	3.17%	11	2
SPY	0.51%	0.67%	0	0

Figure 9. Window returns, average daily swings, and alert-day counts for NVDA, TSLA, and SPY.

View 2: Interactive Alert Grid at 3%

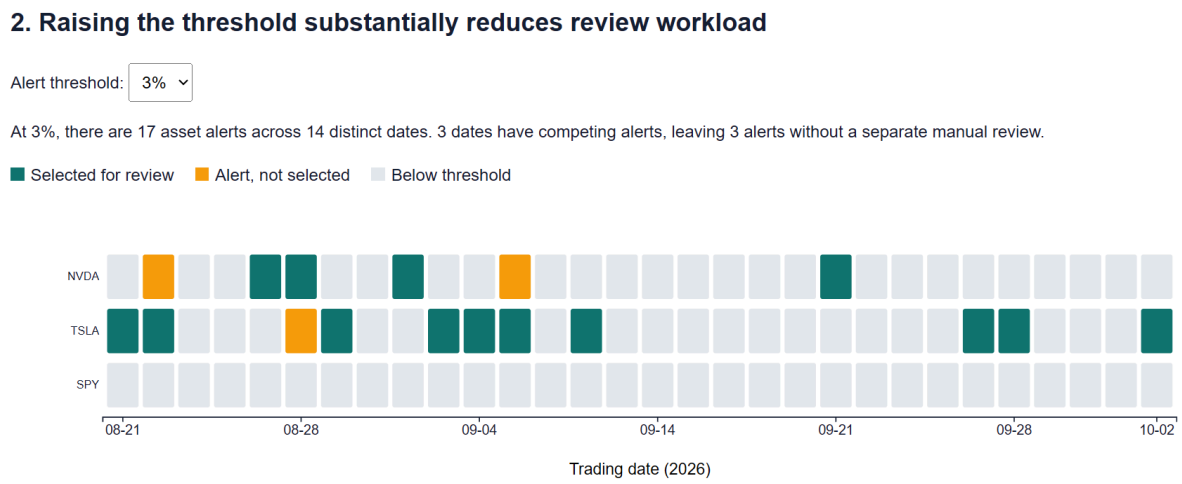


Figure 10. The 3% threshold produces seventeen individual alerts across fourteen trading dates, with three dates containing simultaneous alerts.

View 3: Interactive Alert Grid at 5%



Figure 11. The 5% threshold produces three alerts on three separate dates, with no simultaneous alerts.

The interactive grid distinguishes stocks selected for manual review, alerting stocks not selected, and stocks that remain below the threshold.

Changing the threshold updates both the grid and its associated workload statistics.

2.4 Data Story

Which Stock Should a Risk Manager Review?

A risk-monitoring manager responsible for NVDA, TSLA, and SPY can review only one asset manually per trading day. That constraint makes alert volume a scheduling decision rather than just a statistical count. Using 30 trading dates from August 21 through October 2, 2026, this analysis evaluates whether a 3% or 5% daily price-swing threshold identifies a useful set of after-close reviews. The proposed rule assigns the single daily review to the alerting stock with the largest swing, breaking exact ties alphabetically.

First, the asset with the strongest window return was not the one generating the greatest monitoring workload. NVDA rose 8.96% between its first and last closing prices and had an average daily swing of 2.37%. TSLA returned 2.13% but experienced a larger average daily swing of 3.17%, triggering 11 days at 3%, versus six for NVDA. SPY returned 0.51%, averaged a 0.67% swing, and triggered neither threshold. Monitoring priorities therefore cannot be inferred reliably from the start-to-end return alone: a daily swing captures the intraday high-low range relative to Open rather than a close-to-close gain or loss.

Second, choosing a threshold changes both the amount of work and the breadth of the monitoring screen. The 3% rule produces 17 asset alerts across 14 distinct dates, while 5% produces only three alerts across three dates. Thus, the tighter threshold reduces the number of review dates by 11 of 14, or 78.6%, but also removes 14 lower-threshold asset alerts. The interactive daily grid shows this difference more clearly than totals alone because it preserves when each alert occurred.

The one-review-per-day limit matters on three specific 3% dates: August 24, August 28, and September 8, each with two alerting assets. Under the largest-swing-first rule, TSLA would be selected on August 24 and September 8, and NVDA on August 28. The other three asset alerts remain logged without separate same-day review. At 5%, no dates have competing alerts. The most extreme observations are TSLA at 6.27% on August 31 and 5.60% on August 21, and NVDA at 5.48% on August 28.

I recommend piloting the 3% threshold as an after-close monitoring screen because it identifies a wider set of unusual trading sessions while never scheduling more than one manual review on a date. The trial should record how often the selected review produces useful follow-up and whether bypassed alerts later appear important. If the resulting workload proves unmanageable or unproductive, 5% is a transparent, stricter alternative. This analysis uses only 30 days and does not establish that swing size predicts loss. Daily OHLC data do not reveal when an intraday threshold was crossed, and portfolio exposures, news, and liquidity are absent; those inputs would be necessary before treating the rule as a live risk-control system.

2.5 Source and Reproducibility

Provider: Yahoo Finance through yfinance.

Retrieval timestamp: 2026-10-21T17:53:49.533171+00:00

Library version: 0.2.66

Price policy: Provider Open, High, Low, and Close fields under the specified download settings; Adj Close is excluded from the analysis, and no additional adjustment is applied.

Scope: Thirty trading dates for each of NVDA, TSLA, and SPY.

Saved files: Original provider CSVs, daily observations, calculated metrics, alert records, workload summaries, provenance information, the Colab notebook, and the D3 webpage source.

These data describe price movements within a limited historical window and do not represent a dividend-adjusted total-return series or an operational trading recommendation.

Q3: Childhood Asthma Emergency-Department Visits in New York City

Why I Chose This Dataset

I wanted to explore a health-related issue where data could help explain a real problem and support a practical decision. Since moving to New York City for graduate school, I've become more interested in how different neighborhoods within the same city can have very different experiences.

Childhood asthma caught my attention because it connects public health with access to healthcare and the conditions children experience in their everyday environments. I was particularly interested in emergency-department visits because they represent a measurable healthcare-use event. Instead of examining asthma prevalence alone, I wanted to understand where children were requiring emergency care and whether that need was distributed evenly across the city.

The NYC Department of Health publishes childhood asthma ED-visit data at the community-district level. This made the dataset especially useful because I could compare neighborhoods while considering both population-adjusted rates and the total number of visits.

These two measures answer different questions. A high rate may identify a district with unusually frequent emergency-department use relative to its population, while a high count may indicate considerable absolute demand for healthcare services.

My objective was to determine whether the data could help public-health planners identify communities worth investigating more closely, rather than simply producing a ranking of neighborhoods.

3.1 Audience, Focused Question, and Big Idea

3.1.1 Audience: New York City Department of Health and Mental Hygiene planners responsible for assessing childhood asthma-prevention needs and identifying potential community partnerships.

3.1.2 Focused Question: Which NYC community districts should receive priority for further childhood asthma prevention assessments when both emergency-department visit rates and total visit volume are considered?

3.1.3 Big Idea: Because a few districts have exceptionally high childhood asthma ED-visit rates while all 12 Bronx districts exceed the citywide benchmark, planners should investigate targeted district-level prevention needs alongside a coordinated Bronx-focused assessment.

3.2 Dataset Description and Preparation

Data Source

The dataset was downloaded from the New York City Department of Health and Mental Hygiene's Environment & Health Data Portal.

Dataset: Asthma emergency department visits (age 5 to 17)

Geographic level: Community District

Time period: 2023

Coverage: 59 community districts across the five NYC boroughs

Underlying healthcare data source: New York State SPARCS de-identified hospital discharge data, as identified by the NYC Health Portal.

The dataset contains the following relevant variables:

Variable	Description
TimePeriod	Observation year
GeoID	Community-district identifier
Borough	NYC borough
Geography	Community-district name
Number	Reported ED-visit count
Estimated annual rate per 10,000	Estimated annual ED-visit rate per 10,000 children aged 5–17

Data Cleaning

I imported the downloaded CSV into Google Colab using pandas.

The cleaning process involved renaming columns, converting visit counts and rates into numerical values, and checking for missing observations.

I validated that the dataset contained 59 unique community-district identifiers, covered all five boroughs, and referred exclusively to the 2023 observation year.

I then calculated each district's rate relative to the published NYC overall rate of 143.7 ED visits per 10,000 children.

For the borough comparison, I calculated the minimum, median, and maximum community-district rates within each borough.

These medians describe the distribution of district-level estimates. They should not be confused with population-weighted borough-wide rates.

For the rate-versus-volume analysis, I identified districts exceeding both the citywide rate of 143.7 and the median district visit count of 219.

These reference values support exploratory comparisons; they are not clinical thresholds.

3.3 Evidence Visualization 1: Neighborhood Rate Rankings

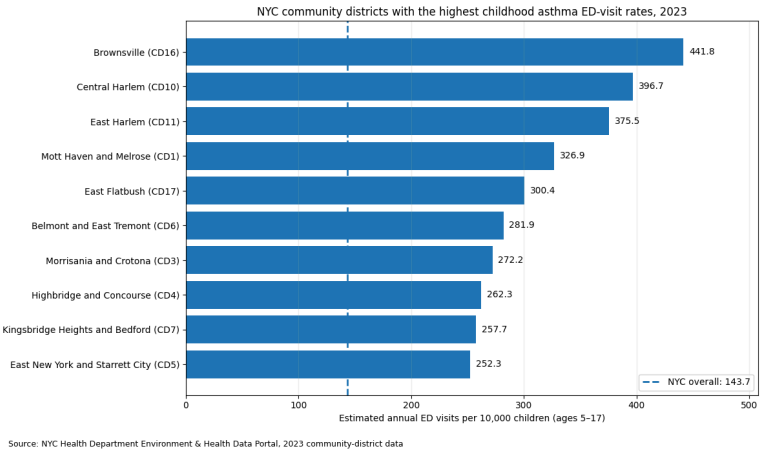


Figure 12. NYC community districts with the highest estimated annual childhood asthma ED-visit rates in 2023. The dashed line represents the citywide benchmark of 143.7 visits per 10,000 children.

The ranked horizontal bar chart identifies districts with especially high rates while preserving readable neighborhood names. Brownsville recorded the highest estimated rate at 441.8 visits per 10,000 children, approximately 3.1 times the citywide rate.

Horizontal bars allow direct comparison of district rates while keeping long district names readable; the citywide reference line shows how far each estimate exceeds the benchmark.

Central Harlem recorded 396.7, and East Harlem recorded 375.5. Both Harlem districts exceeded 2.6 times the citywide benchmark. This visualization establishes that the highest-rate communities differ substantially from the citywide level.

Evidence Visualization 2: Borough-Level Differences

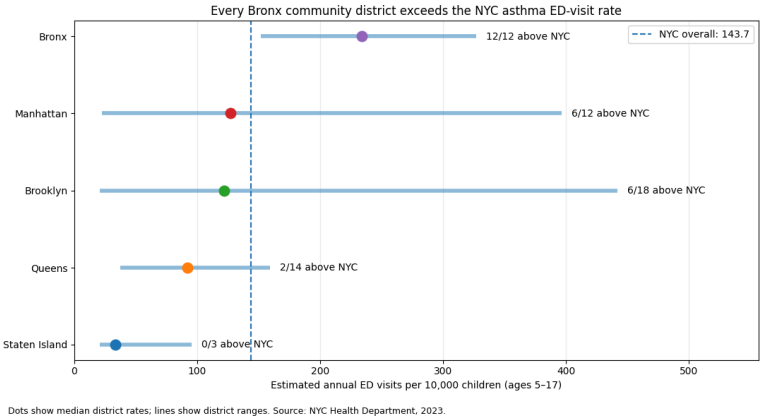


Figure 13. Minimum, median, and maximum community-district ED-visit rates within each NYC borough. The chart also identifies how many districts exceed the citywide benchmark.

The borough comparison provides a broader perspective on the distribution of asthma-related ED visits.

The results show:

Borough	Districts	Median District Rate	Districts Above NYC Rate

Bronx	12	234.10	12
Manhattan	12	127.10	6
Brooklyn	18	121.90	6
Queens	14	92.05	2
Staten Island	3	33.60	0

All twelve Bronx community districts exceed the citywide benchmark. The lowest Bronx district rate is 151.8, which is still above NYC's overall rate of 143.7. Brooklyn and Manhattan contain individual high-rate districts but also show substantial variation. This comparison distinguishes isolated high-rate neighborhoods from a pattern affecting an entire borough.

Showing each borough's district minimum, median, and maximum distinguishes a broadly elevated distribution from a borough containing only a few exceptionally high-rate districts.

Evidence Visualization 3: Rate Versus Visit Volume

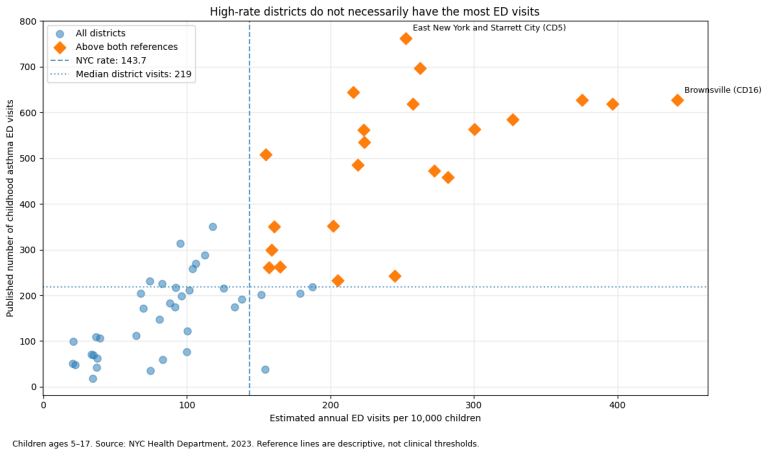


Figure 14. Relationship between population-adjusted ED-visit rates and absolute visit counts across NYC's 59 community districts. Reference lines indicate the citywide rate and median district visit count.

This chart addresses the distinction between relative healthcare use and absolute service demand. Brownsville recorded 627 visits at a rate of 441.8 per 10,000 children. East New York and Starrett City recorded 763 visits at a lower rate of 252.3. Highbridge and Concourse recorded 697 visits at a rate of 262.3.

Plotting each district's rate against its visit count lets planners compare relative frequency and absolute volume simultaneously.

The analysis identified 22 districts exceeding both the citywide rate and the median district visit count.

These districts provide candidates for further investigation because they combine relatively elevated ED-visit rates with substantial reported visit volumes.

3.4 Written Data Story

Where Should NYC Prioritize Childhood Asthma Prevention?

A citywide asthma statistic can conceal where families may be facing the greatest demand for emergency care. For children aged 5–17, the NYC Health Department reported an estimated 143.7 asthma emergency-department (ED) visits per 10,000 children in 2023. To help public-health planners decide where further prevention work might be most useful, I compared this benchmark with data covering all 59 community districts. I examined both rates, which make population comparisons possible, and visit counts, which show the absolute volume of healthcare use.

Brownsville stands out in the ranked district chart. Its estimated rate was 441.8 per 10,000, approximately 3.1 times the citywide benchmark. Central Harlem followed at 396.7 and East Harlem at 375.5, each more than 2.6 times the citywide rate. These differences are large enough to warrant closer assessment of local care access and prevention opportunities. A high rate does not prove that asthma is more common in exactly the same proportion of children: ED-visit rates measure visits rather than the number of unique children affected.

The borough comparison adds a different kind of evidence. Every one of the Bronx's 12 community districts exceeded the citywide rate. The Bronx median district rate was 234.1, and even its lowest district measured 151.8. Brooklyn and Manhattan contain some extreme districts, but only six of 18 Brooklyn districts and six of 12 Manhattan districts were above the city benchmark. Queens had two of 14 above it, and Staten Island had none of three. This suggests a distinction between a concentrated need in specific districts and a broadly elevated pattern across an entire borough. The plotted medians summarize district rates; they are not borough-wide population-weighted estimates.

The third chart changes the conversation from relative burden to service volume. Brownsville recorded 627 visits, but East New York and Starrett City recorded 763 at a lower estimated rate of 252.3. Highbridge and Concourse recorded 697 visits at 262.3. Therefore, concentrating solely on the highest rates could overlook districts with substantial numbers of ED encounters. Across NYC, 22 districts exceeded both the 143.7 citywide-rate reference and the median district count of 219. This is an exploratory way to identify places needing closer assessment, not an official prioritization formula.

Taken together, the findings support investigating a small set of especially high-rate districts, including Brownsville and Central and East Harlem, while also assessing whether the Bronx needs a coordinated borough-focused response. The review should involve local clinics, schools, and community organizations to identify feasible prevention and follow-up opportunities. High-volume locations such as East New York deserve separate attention when considering service capacity.

The evidence has important limits. This is a one-year, district-level analysis of healthcare use, not a study of asthma causation or individual patients. Repeat visits can affect counts; severity, treatment access, insurance, local environments, and care-seeking patterns may all matter. The file does not include uncertainty intervals, the child-population denominators used to calculate the rates, or direct measures of these potential drivers. I would validate current patterns with newer years and additional local data before recommending funding amounts or declaring a district's risk definitively higher than another's.

3.8 Limitations and Future Work

The analysis identifies geographic differences in asthma-related ED use but does not establish their underlying causes.

Several limitations affect how strongly the results can be interpreted.

First, emergency-department visits are not equivalent to the number of unique children with asthma. A child may contribute multiple visits during the observation period.

Second, healthcare utilization can reflect several factors, including asthma severity, access to primary care, availability of preventive treatment, and differences in healthcare-seeking behavior.

Third, the analysis uses a single year of community-district data. A longer historical period would help determine whether the observed differences are persistent.

Fourth, the provided CSV does not include uncertainty intervals. Small differences between districts should therefore not be treated as definitive statistical differences.

Future work could combine asthma ED-visit data with measures of housing quality, air pollution, access to routine healthcare, and additional years of observations.

Such an analysis could help planners identify possible contributing factors and assess whether targeted prevention programs should be expanded.

3.9 Data Source and Reproducibility

Primary source: NYC Department of Health and Mental Hygiene, Environment & Health Data Portal.

Dataset: Asthma emergency department visits, ages 5–17.

Time period: 2023.

Geographic coverage: 59 NYC community districts.

Source URL: https://a816-dohbesp.nyc.gov/IndicatorPublic/data-explorer/asthma/

Access date: October 21, 2026.

Original data file: asthma_recent_original.csv

Cleaned data: asthma_2023_cleaned.csv

Analysis file: asthma_2023_analysis.csv

Borough summary: borough_summary.csv

Candidate districts: priority_assessment_candidates.csv

Analysis notebook: HW3_Q3_Childhood_Asthma.ipynb

Visualizations: q3_asthma_top10.png, q3_borough_comparison.png, q3_rate_vs_visits.png

The included notebook documents the cleaning, calculations, and visualization procedures needed to reproduce the results.

References

1. Vega. *Seattle Weather Dataset*. https://raw.githubusercontent.com/vega/vega/main/docs/data/seattle-weather.csv

2. D3.js. *Data-Driven Documents, Version 7*. https://d3js.org/
3. Massive. *Stocks API: Custom Bars*. https://massive.com/docs/rest/stocks/aggregates/custom-bars
4. yfinance. *Yahoo Finance Market Data Downloader*. https://pypi.org/project/yfinance/
5. NYC Department of Health and Mental Hygiene. *Environment & Health Data Portal: Asthma*. https://a816-dohbesp.nyc.gov/IndicatorPublic/data-explorer/asthma/
6. NYC Department of Health and Mental Hygiene. *Bronx Asthma Case Management Initiative Concept Report* (2025). https://www.nyc.gov/assets/doh/downloads/pdf/acco/2025/bx-case-management-initiative-concept-report.pdf
7. New York State Department of Health. *Statewide Planning and Research Cooperative System (SPARCS)*.